

Python Básico

II. Objetos

Objetos compostos unidimensionais (vetores)

tupla, lista, dicionário e set

Conclusões – Objetos Simples

- Readability
- Beautiful is better than ugly.
- Object-oriented programming (OOP)
- `nome_objeto` recebe valor
- Escopo
- Nomes de variáveis
- IDE



Objetos Simples

Texto: Cadeia de caracteres alpha numéricos

str: "O valor é R\$ 11,00" 'A luz está "apagada"'

Inteiros: Números inteiros

int: 42, 0, 1, 25, 3471

Flutuantes: Números decimais

float: 4.2, 0.34, 225.0, 1.1, 5.0

Booleana: Valores binários (lógicos)

bool: False, True

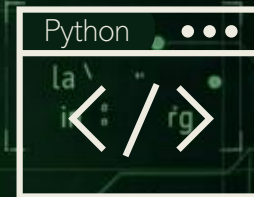
Complexo: número complexos (real imaginário)

complex: 24 + 7j

Bytes: cadeia de caracteres em *ASCII

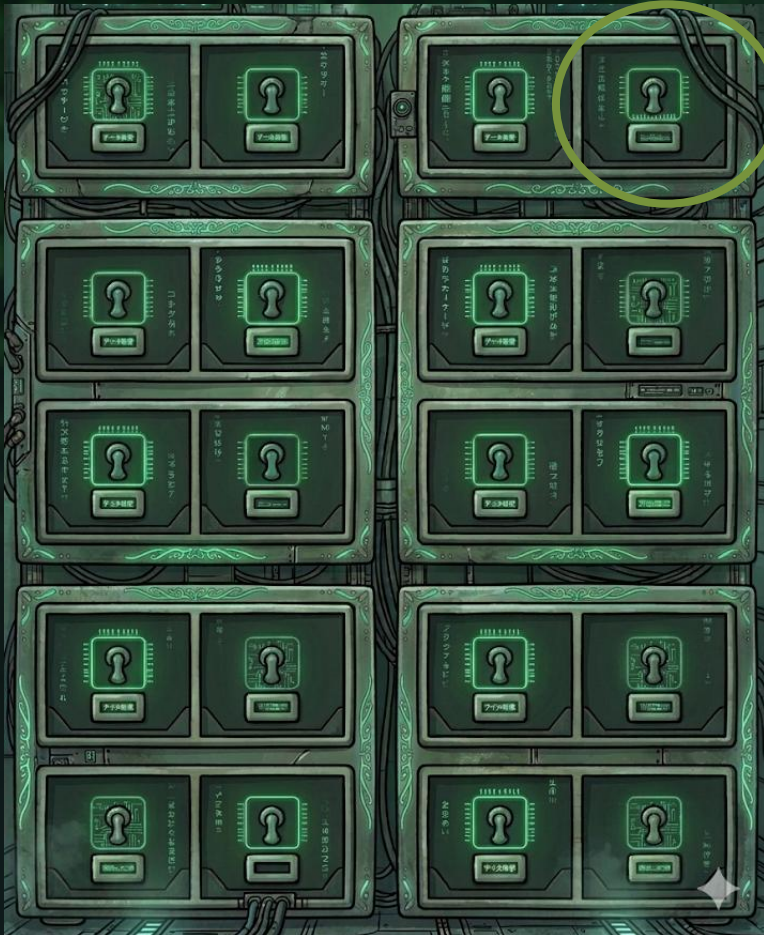
bytes: b'\x00\x00\x00\x00\x00'

* American Standard Code for Information Interchange



Random-Access Memory – RAM

RAM



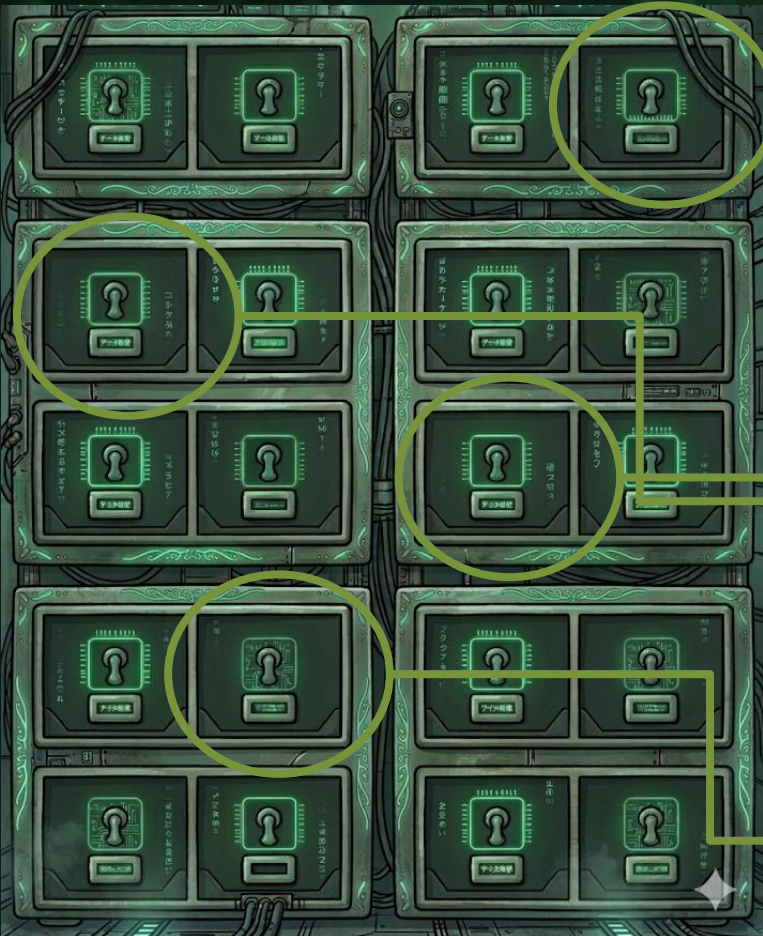
```
1 lampada = "ligada"
2 # Obtém o ID (inteiro)
3 identificador = id(lampada)
4 print(f"Id: {identificador}")
5
6 # Obtém o endereço em hexadecimal (formato de memória)
7 endereco_memoria = hex(id(lampada))
8 print(f"End hexa: {endereco_memoria}")
9
```

[2] ✓ 0.0s

... Id: 138631344823280
End hexa: 0x7e15a0101bf0

Objetos Simples

RAM



```
1  lampada = "ligada"  
2  lamp_1 = "desligada"  
3  lamp_2 = "desligada"  
4  lamp_3 = "ligada"
```

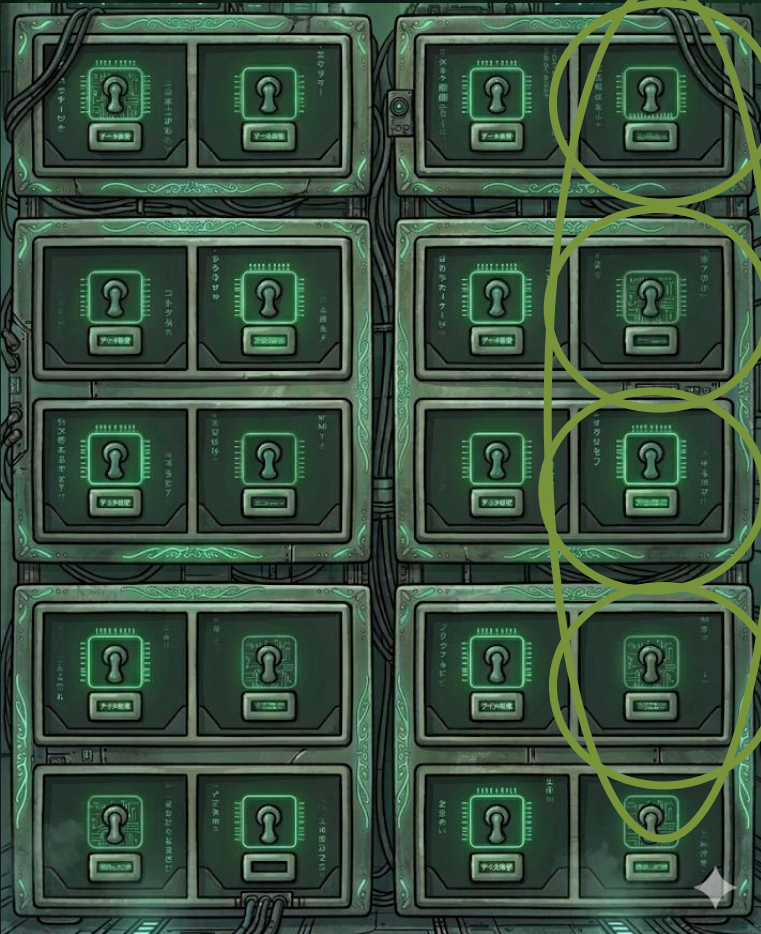
[3] ✓ 0.0s



Objetos Compostos

RAM

lamps



```
1 lamps = [  
2     "ligada",  
3     "desligada",  
4     "desligada",  
5     "ligada",  
6 ]  
[5] ✓ 0.0s
```



Instanciando Objetos Compostos

nome_objeto_simples receber valor

```
1 lampada = "ligada"
```

nome_objeto_composto recebe valor[index], valor[index], ... valor[index]

nome_objeto_composto[index]

```
1 lamps = [  
2     "ligada",  
3     "desligada",  
4     "desligada",  
5     "ligada",  
6 ]  
[5] ✓ 0.0s
```

```
1 print(lamps[0])  
[7] ✓ 0.0s  
... ligada
```

nome_objeto_composto[index].nome_função(parâmetro[opcional])

```
1 lamps[0].count("a")  
[8] ✓ 0.0s  
... 2
```



Objetos Compostos mult classe

nome_objeto_composto recebe valor[index], valor[index], ... valor[index]

```
1 objs_simples = {
2     "str": "ligada",
3     "int": 42,
4     "float": 2.25,
5     "bool": True,
6     "complex": 1 + 2j,
7     "byte": b"Python"
8 }
9 objs_simples
```

[11] ✓ 0.0s

```
... {'str': 'ligada',
      'int': 42,
      'float': 2.25,
      'bool': True,
      'complex': (1+2j),
      'byte': b'Python'}
```



Objetos Compostos -> delimitadores

Tupla: () parênteses

Lista: [] colchetes

Dicionário: {} chaves

Set: {} chaves

Array: NumPy

DataFrame: Pandas



Objetos Compostos -> syntax

Tupla: nome_tupla recebe (valor, valor, valor)

```
1 nome_tupla = ("valor 1", "valor 2", "valor 3")
```

Lista: nome_lista recebe [valor, valor, valor]

```
3 nome_lista = ["valor 1", "valor 2", "valor 3"]
```

Dicionário: nome_dicionario recebe {index: valor, index: valor}

```
5 nome_dicionario = {"index 1": "valor 1", "index 2": "valor 2", "index 3": "valor 3"}
```

Set: nome_set recebe {valor, valor, valor}

```
7 nome_set = {"valor 1", "valor 2", "valor 3"}
```



Objetos Compostos -> características

Auto indexada
Imutável

```
1 nome_tupla = ("valor 1", "valor 2", "valor 3")
```

Auto indexada
Mutável

```
3 nome_lista = ["valor 1", "valor 2", "valor 3"]
```

Index manual
Mutável

```
5 nome_dicionario = {"index 1": "valor 1", "index 2": "valor 2",
```

Não index
Imutável

```
7 nome_set = {"valor 1", "valor 2", "valor 3"}
```



Tuplas e strings

tupla

```
1 nome_tupla = ("P", "y", "t", "h", "o", "n")
2 nome_tupla
```

[13] ✓ 0.0s

... ('P', 'y', 't', 'h', 'o', 'n')

```
1 nome_tupla = ("P", "y", "t", "h", "o", "n")
2 nome_tupla[1:4]
```

[14] ✓ 0.0s

... ('y', 't', 'h')

string

```
1 nome_string = "Python"
2 nome_string
```

[15] ✓ 0.0s

... 'Python'

```
1 nome_string = "Python"
2 nome_string[1]
```

[16] ✓ 0.0s

... 'y'

```
1 nome_string = "Python"
2 nome_string[1:4]
```

[17] ✓ 0.0s

... 'yth'



Conclusões

- RAM
- Objetos Simples
- Objetos Compostos
- Valor e index
- `nome_objeto_composto[index]`
- Diferentes Objetos Compostos





Vamos
praticar?

LIMC-IA



Vamos
exercitar?

LIMC-IA



Exercícios

01 – Quais as vantagens da utilização de objetos compostos em detrimento dos objetos simples?

02 – Instancie uma variável da classe tupla, contendo 5 valores de qualquer classe, e recupere os valores:

03 – Instancie uma variável da classe lista, contendo 5 valores de qualquer classe, e recupere cada um dos valores:



Exercícios

04 – Instancie uma variável da classe lista, contendo 5 nomes de genes diferentes, recupere de cada um dos valores quantas letras 'a' estão presentes:

05 – Instancie uma variável da classe lista, contendo 5 valores, e recupere do segundo ao quarto valores:

06 – Instancie uma variável da classe string com o seu nome como valor, e recupere o segundo e o último valores:



Exercícios

07 – Instancie uma variável da classe dicionário com quantos valores inteiros desejar. Usando as funções globais, retorne a classe do objeto, quantos valores estão instanciados e a soma dos valores contidos no objeto:

08 – Instancie uma variável da classe dicionário. Os index dos valores devem ser os valores contidos na lista instanciada no exercício 4 (Nomes de genes). Para isso você deve utilizar a indexação de lista. Os valores devem ser o número de caracteres em cada index. Retorne o objeto:



Exercícios

09 – Instancie cinco variáveis da classe lista. Cada uma delas será referente à um dos valores contidos na lista instanciada no exercício 4 (Nomes dos genes). Em cada uma das listas, você deve inserir o número de caracteres presentes no nome do gene, os dois primeiros caracteres do nome, quantas vezes aparece o caractere 'a' e se no nome aparecem os caracteres 'hu'. Para isso você deve utilizar a indexação de lista. Retorne cada um dos objetos:



Exercícios

10 – Instancie uma variável da classe dicionário, os index dos valores devem ser os valores contidos na lista instanciada no exercício 4 (Nomes dos genes). Para isso você deve utilizar a indexação de lista. Os valores devem ser as listas instanciadas no exercício 9. Retorne o objeto.



Python Básico



21 de março de 2026